Course Notes

Lecture 05 — Bluetooth Low Energy: Programming

Communication Technologies for the Internet of Things

Academic year 2026–2027

Catarina Silva

ESTGA, University of Aveiro

Text to support independent study

Contents

How to Use These Notes **2**

1 Commentary on the Slides **2**

2. Agenda (1/2) . . . 2
3. Agenda (2/2) . . . 3
4. Architecture of the Complete Chain . . . 4
5. Architecture of the Exercise (1/2) . . . 5
6. Architecture of the Exercise (2/2) . . . 6
7. BLE Libraries and Wrappers (1/2) . . . 7
8. BLE Libraries and Wrappers (2/2) . . . 8
9. bluepy: Class Model (1/2) . . . 9
10. bluepy: Class Model (2/2) . . . 10
11. Implementing a Scanner (1/2) . . . 11
12. Implementing a Scanner (2/2) . . . 12
13. Roles and Direction of Exchanges . . . 13
14. Notifications: Concept (1/2) . . . 14
15. Notifications: Concept (2/2) . . . 15
16. GATT Operations and Energy Cost . . . 16
17. Notifications: Implementation in bluepy (1/2) . . . 17
18. Notifications: Implementation in bluepy (2/2) . . . 18
19. The HM-11 BLE Shield (1/2) . . . 19
20. The HM-11 BLE Shield (2/2) . . . 20
21. Structure of the Arduino Sketch . . . 21
22. Structure of the Arduino Program (1/2) . . . 22
23. Structure of the Arduino Program (2/2) . . . 23
24. Verification on the Gateway Side (1/2) . . . 24
25. Verification on the Gateway Side (2/2) . . . 25
26. Recommended Working Method (1/2) . . . 26
27. Recommended Working Method (2/2) . . . 27
28. Lesson Summary (1/2) . . . 28
29. Lesson Summary (2/2) . . . 29
30. Sources . . . 30

Final Check **31**

References **31**

How to Use These Notes

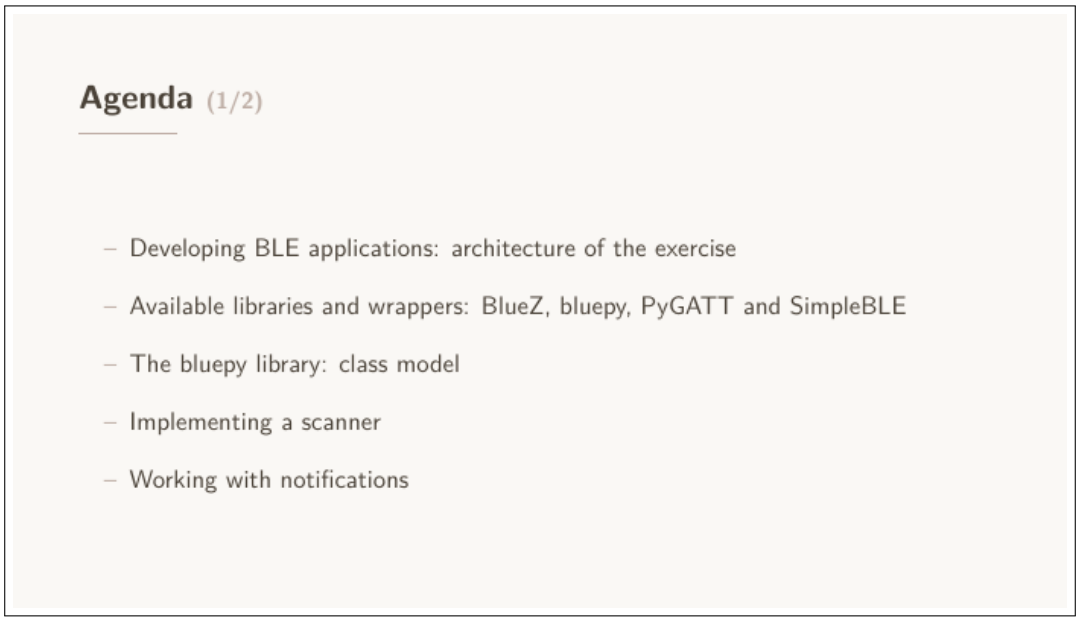
These notes follow the lecture slide by slide. Each page reproduces a slide and states, below it, what should be taken from it.

Reading them does not replace the lecture or the slides. They are meant for later study, once the student has heard the lecture and wants to consolidate, check that the essentials were understood or prepare for assessment. The comments make explicit links between slides that remain implicit in the oral presentation, and point out where the material is commonly misunderstood.

Whenever a comment poses a question, that question is the verification criterion: if you cannot answer it, go back to the slide before moving on.

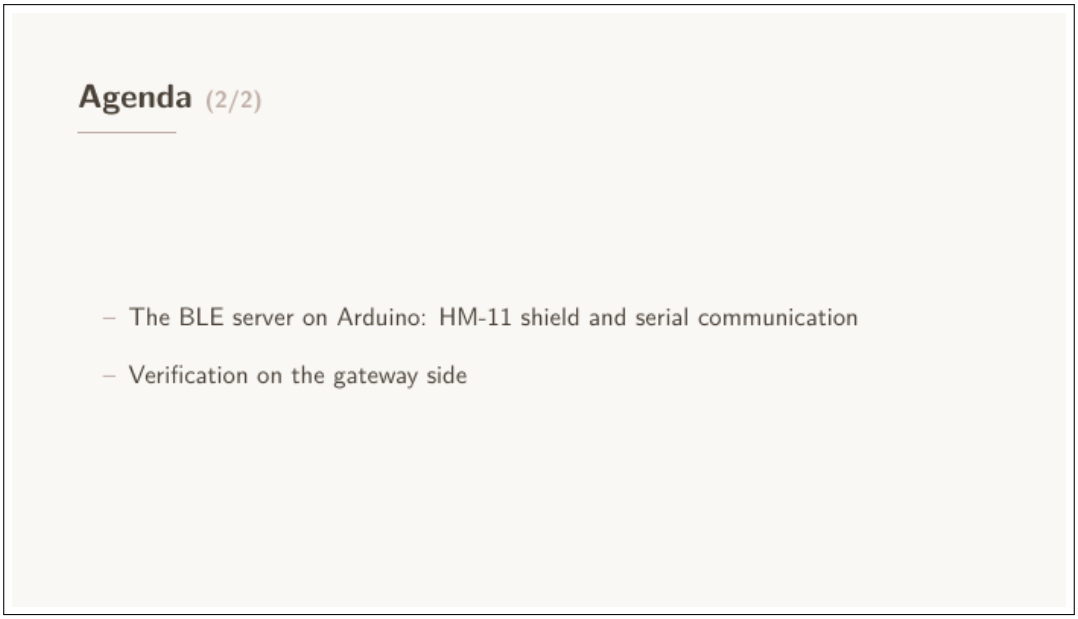
1. Commentary on the Slides

Slide 2. Agenda (1/2)



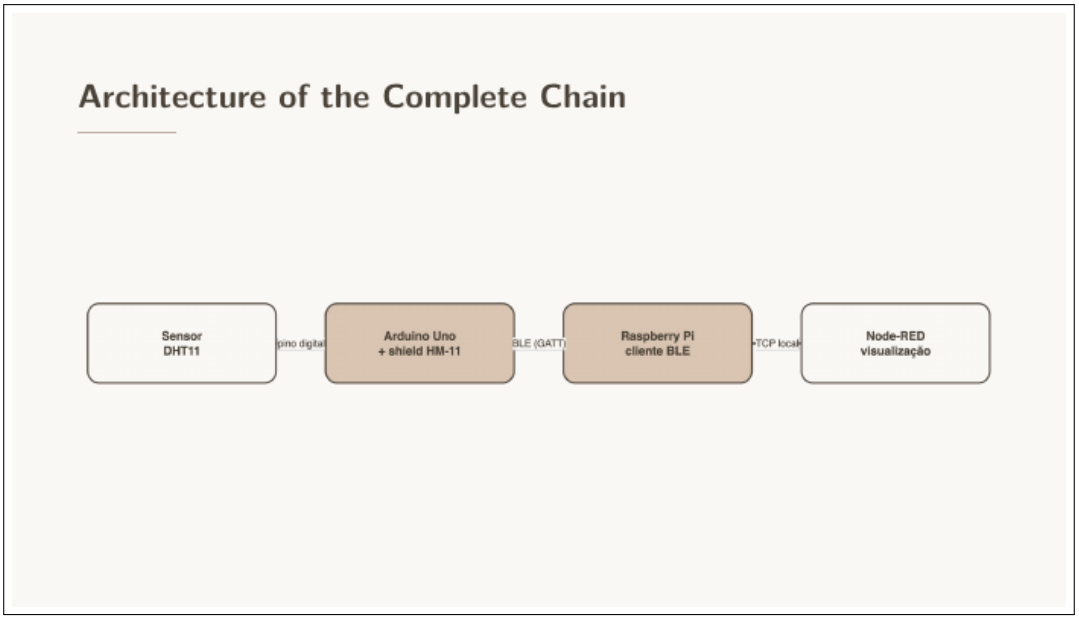
The path through the lecture. The lecture has a top-down structure. It starts from the architecture of the system to be built, goes down to the available libraries, then down to the class model of one of them and ends in concrete code. This order serves a purpose: it prevents the student from starting to write instructions before knowing which pieces are being put together.

Slide 3. Agenda (2/2)



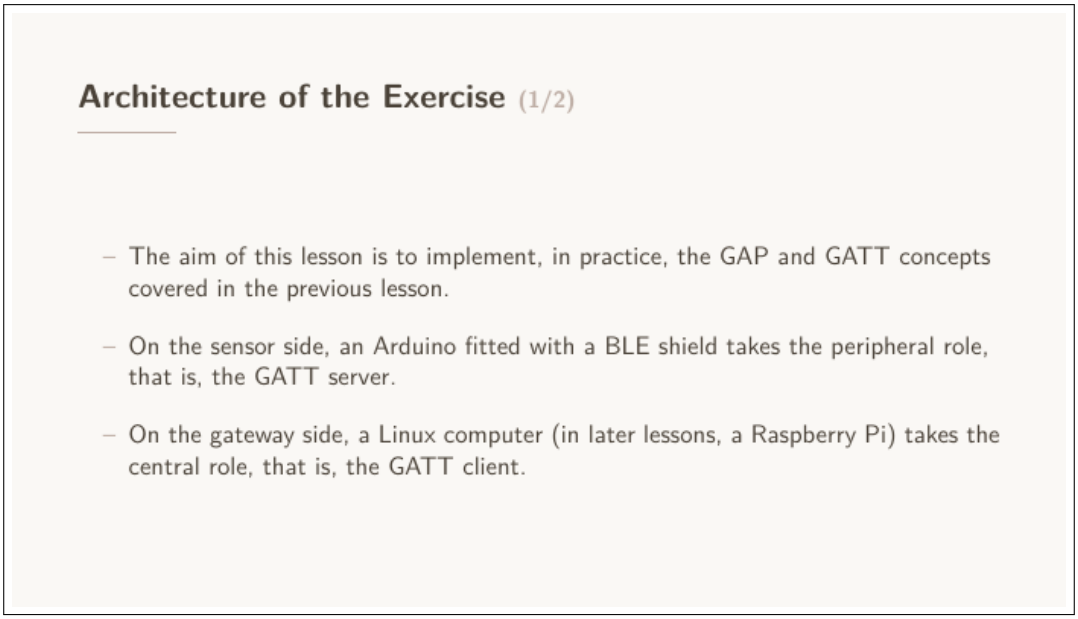
The second half. The last two topics change sides. They move from the client, which is programmed, to the server, which is assembled, and end with verification from the gateway.

Slide 4. Architecture of the Complete Chain



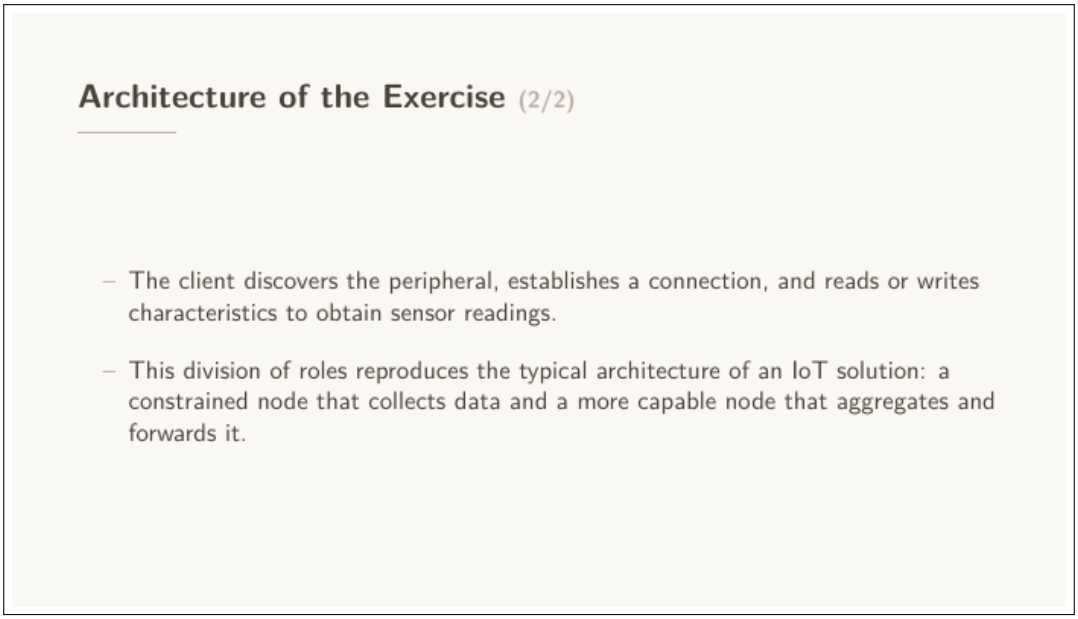
The complete system, all at once. The diagram runs through the four elements of the chain and gives the protocol of each leg. It is worth fixing it before moving on, because all the following slides deal with a single leg at a time. Note that only the first leg is BLE. From the gateway onwards, communication is already over IP.

Slide 5. Architecture of the Exercise (1/2)



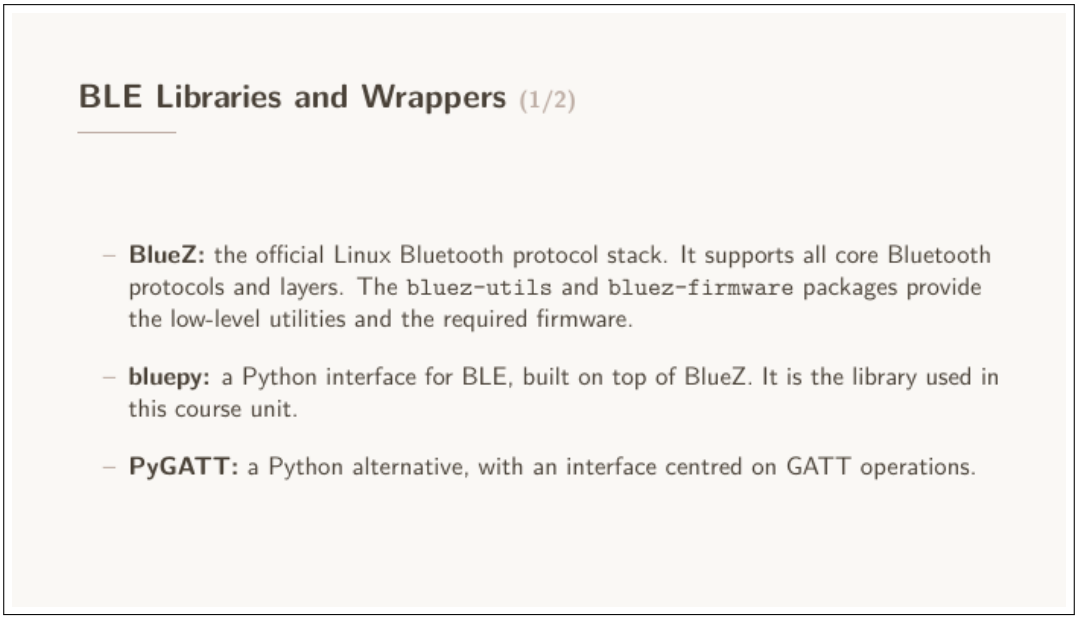
Who does what. The Arduino with the BLE shield takes the role of peripheral, that is, GATT server. The Linux computer takes that of central, that is, GATT client. This correspondence between GAP roles and GATT roles is not mandated by the standard, but it is the usual configuration and the one used throughout the course.

Slide 6. Architecture of the Exercise (2/2)



Why this split is representative. The client discovers, connects and reads or writes characteristics. The split reproduces the typical architecture of an IoT solution: a constrained node that collects data and a capable node that aggregates and forwards it. Recognising this pattern is useful, because it reappears with other technologies throughout the semester.

Slide 7. BLE Libraries and Wrappers (1/2)



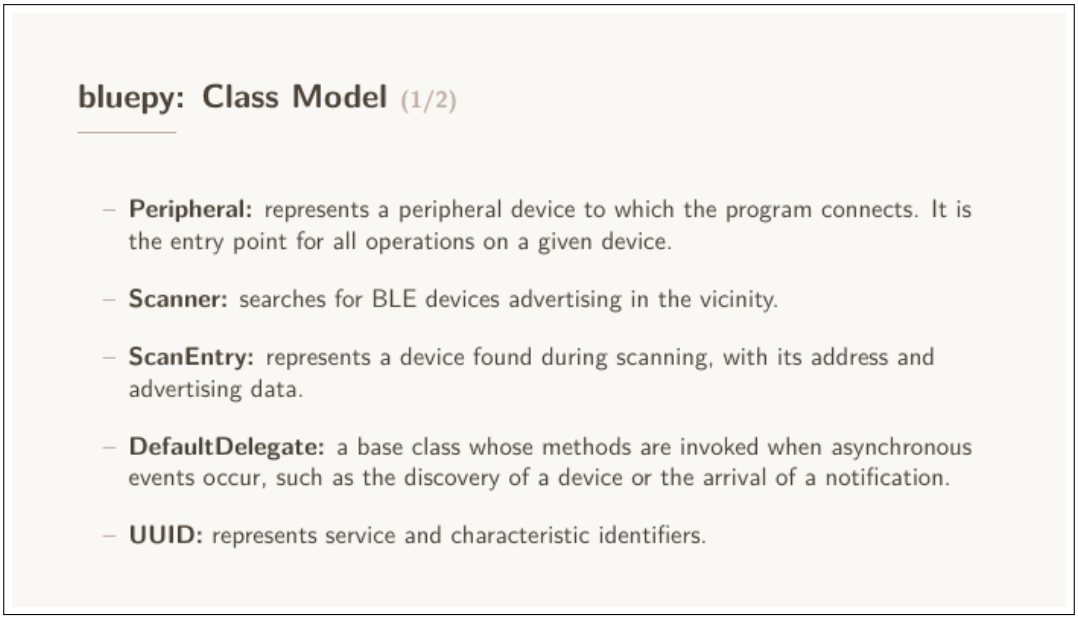
The options available. BlueZ is the official Linux protocol stack and supports all the core Bluetooth protocols. bluepy is a Python interface built on it, and is the library adopted in this course. PyGATT is an alternative focused on GATT operations.

Slide 8. BLE Libraries and Wrappers (2/2)



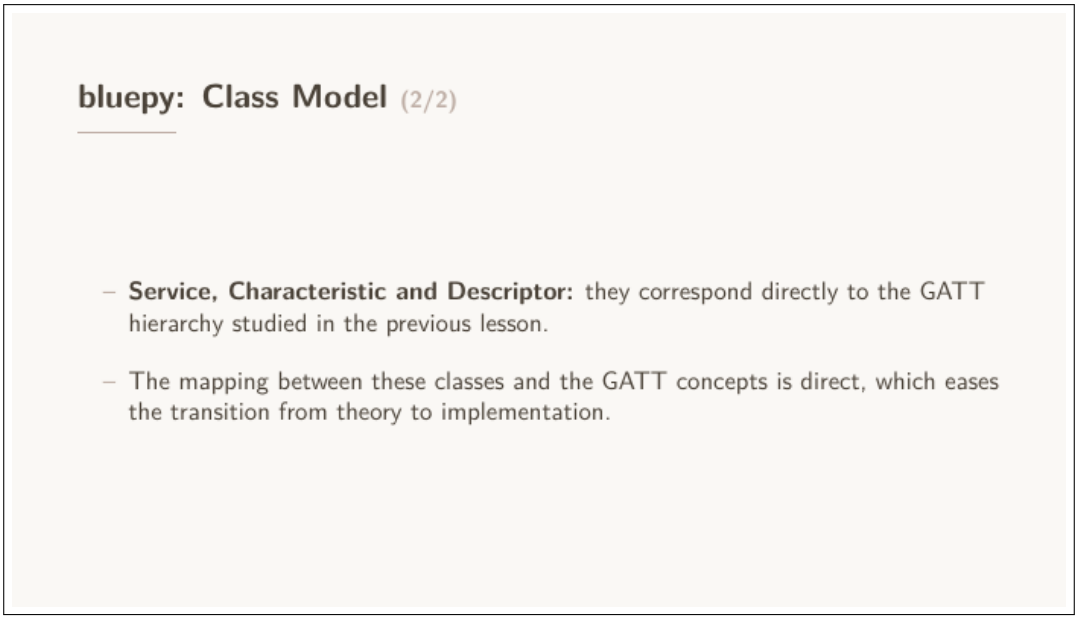
What the libraries have in common. SimpleBLE offers a simplified interface. The observation to retain is the last one: the choice of library conditions the level of abstraction, but on Linux they all sit on the same BlueZ stack. A useful practical consequence follows: if something does not work at library level, it is worth checking first at BlueZ level, with the tools from Lecture 04.

Slide 9. bluepy: Class Model (1/2)



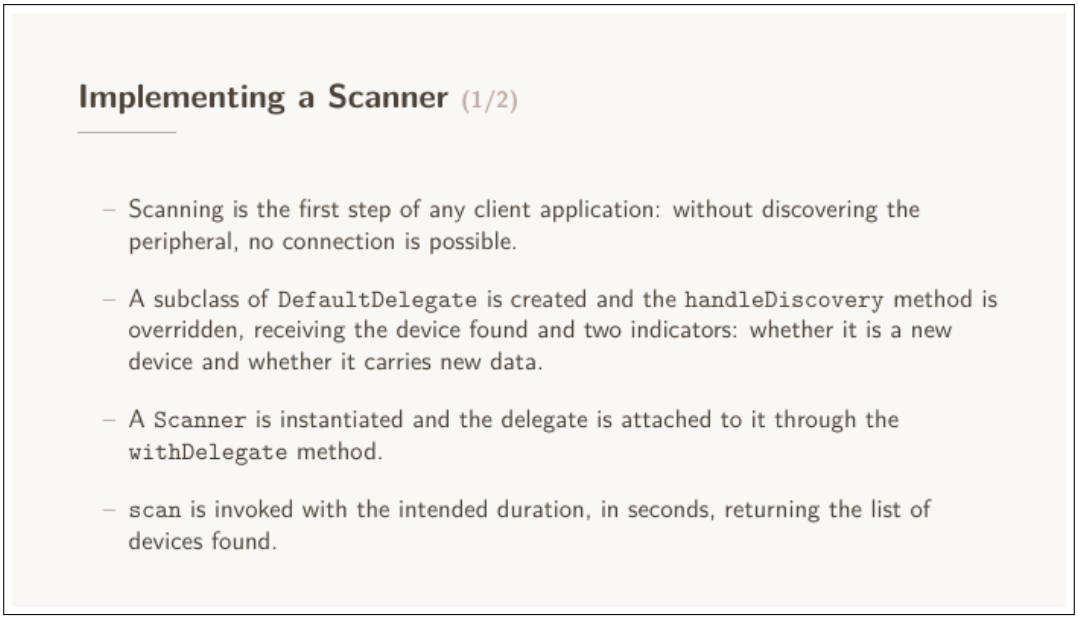
The classes that matter. Peripheral represents a device the program connects to. Scanner looks for devices that advertise. ScanEntry represents a device found. DefaultDelegate is the base class whose methods are invoked when asynchronous events occur. UUID represents the identifiers.

Slide 10. bluepy: Class Model (2/2)



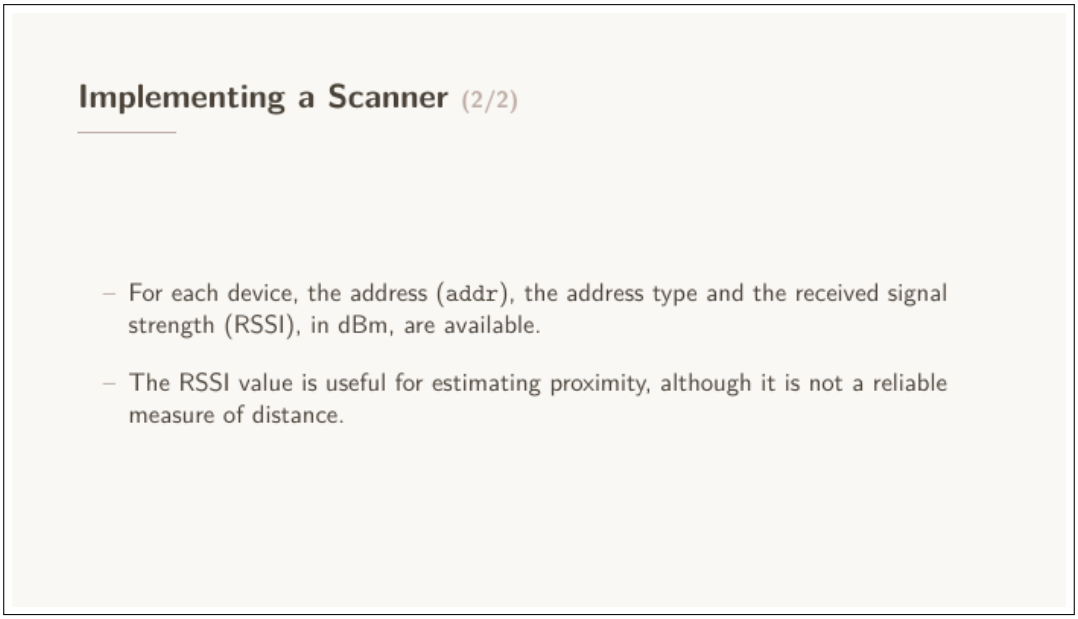
The correspondence with Lecture 04. Service, Characteristic and Descriptor correspond directly to the GATT hierarchy studied in the previous lecture. This direct correspondence is the reason why the theory of Lecture 04 must be consolidated before this one. Whoever masters the attribute hierarchy immediately recognises the class model.

Slide 11. Implementing a Scanner (1/2)



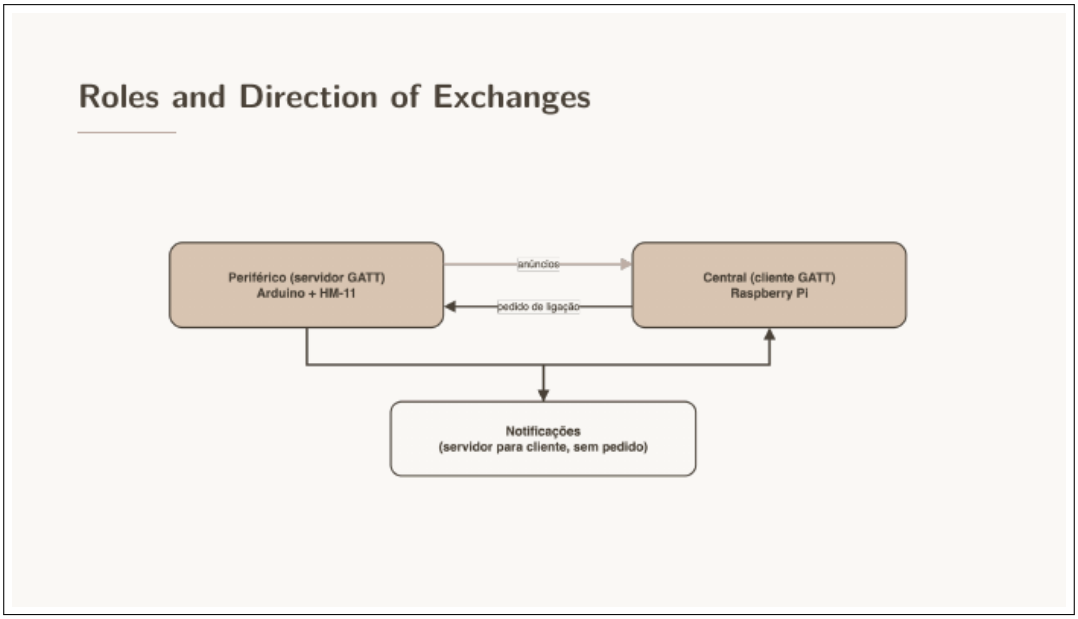
How to implement a scanner. Scanning is the first step of any client application: without discovering the peripheral, no connection is possible. The procedure has four steps. Create a subclass of DefaultDelegate and override handleDiscovery. Instantiate a Scanner. Attach the delegate with withDelegate. Invoke scan with the desired duration.

Slide 12. Implementing a Scanner (2/2)



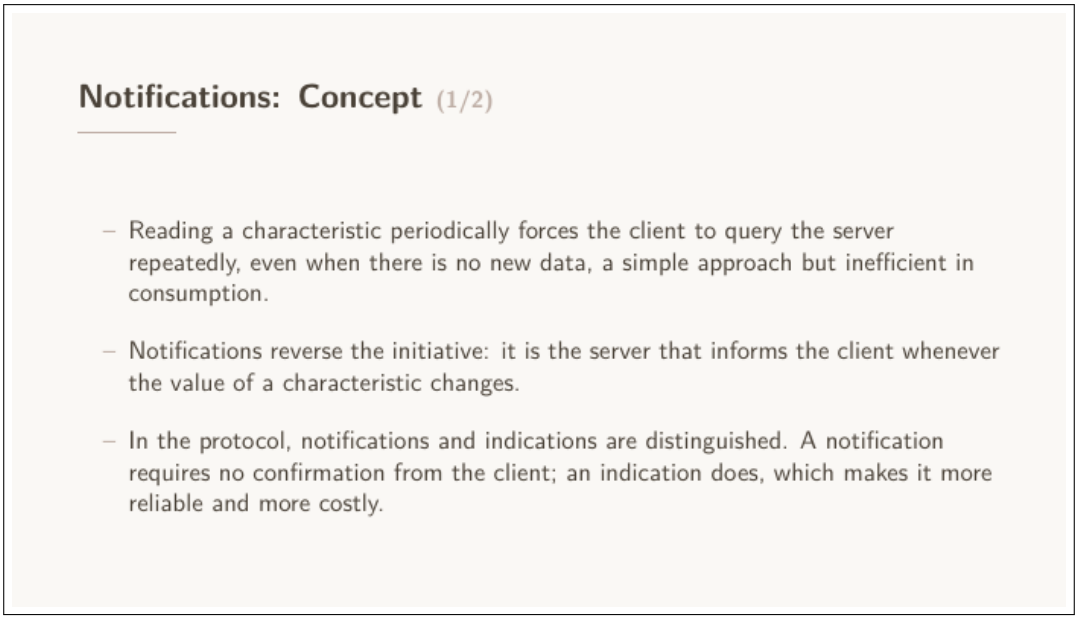
What is obtained from each device. The address, the address type and the received signal strength, in dBm, become available. The warning about RSSI is important: it serves to estimate relative proximity, but is not a reliable measure of distance, because attenuation depends on obstacles and reflections.

Slide 13. Roles and Direction of Exchanges



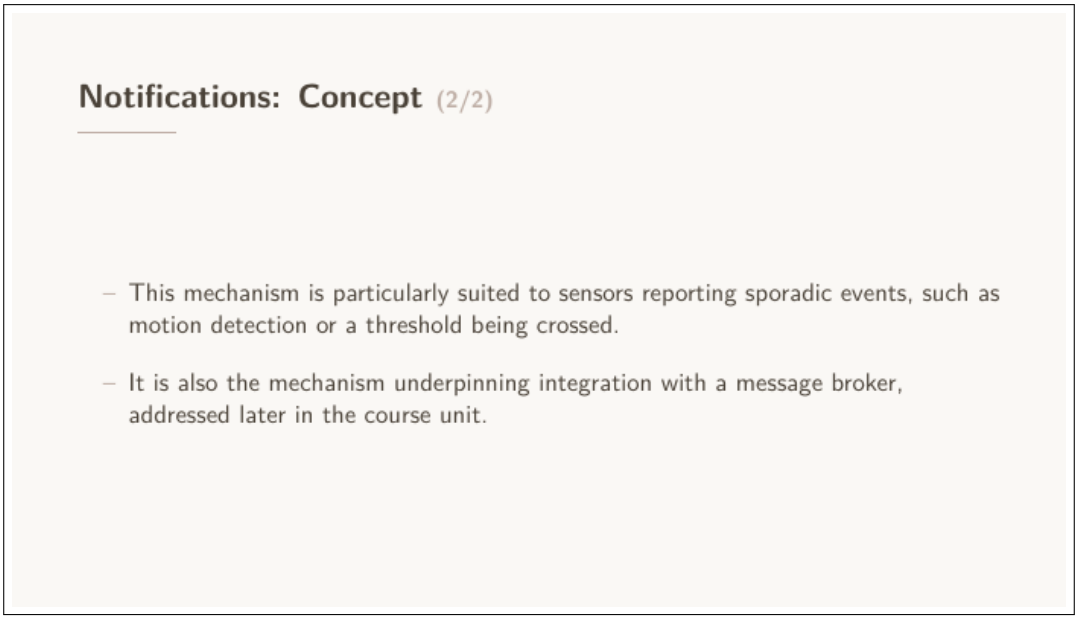
Who initiates each type of exchange. The diagram shows the two directions. Advertisements come from the peripheral. The connection request comes from the central. Notifications, once the connection is established, again come from the peripheral, without the client asking for them each time. It is this last arrow that distinguishes notifications from everything else.

Slide 14. Notifications: Concept (1/2)



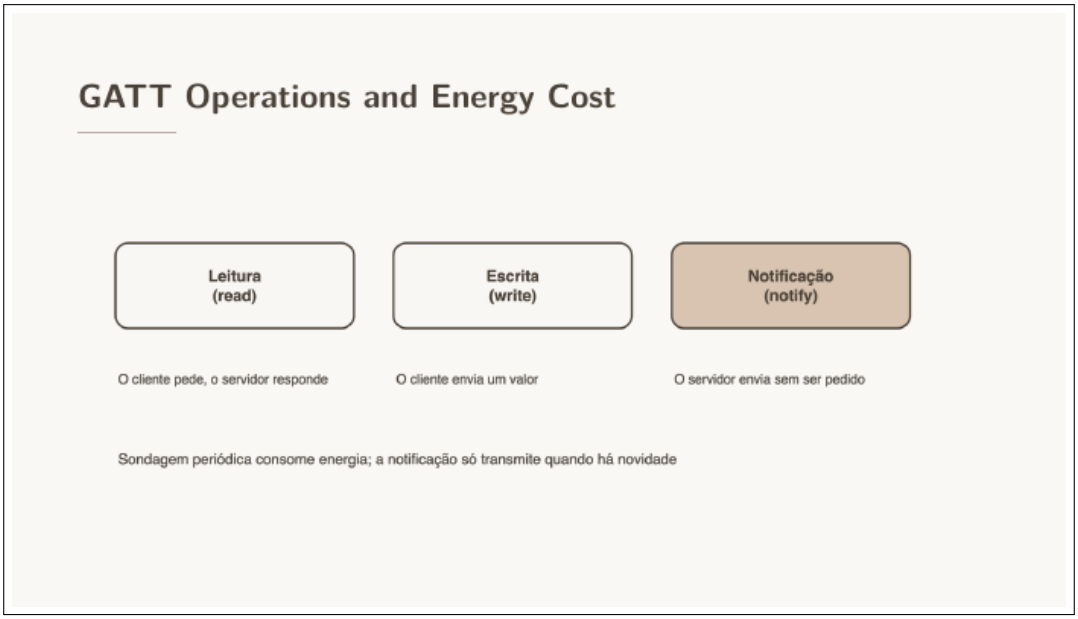
Why polling is inefficient. Periodically reading a characteristic forces the client to repeatedly poll the server, even when there is no new data. Notifications invert the initiative: it is the server that informs when the value changes. The distinction between notification and indication should be retained: the first needs no acknowledgement, the second does, which makes it more reliable and more expensive.

Slide 15. Notifications: Concept (2/2)



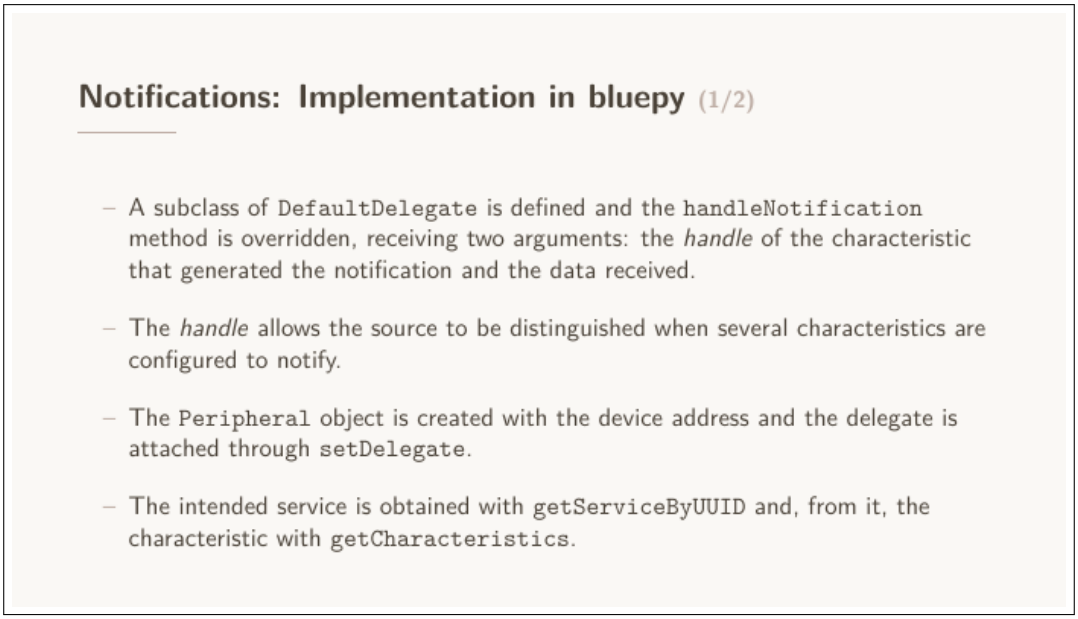
Where the mechanism applies. Notifications are particularly suited to sensors that report sporadic events, such as motion detection or crossing a threshold. They are also the mechanism that sustains integration with a message broker, covered in Lecture 12. It is worth fixing this link from now on.

Slide 16. GATT Operations and Energy Cost



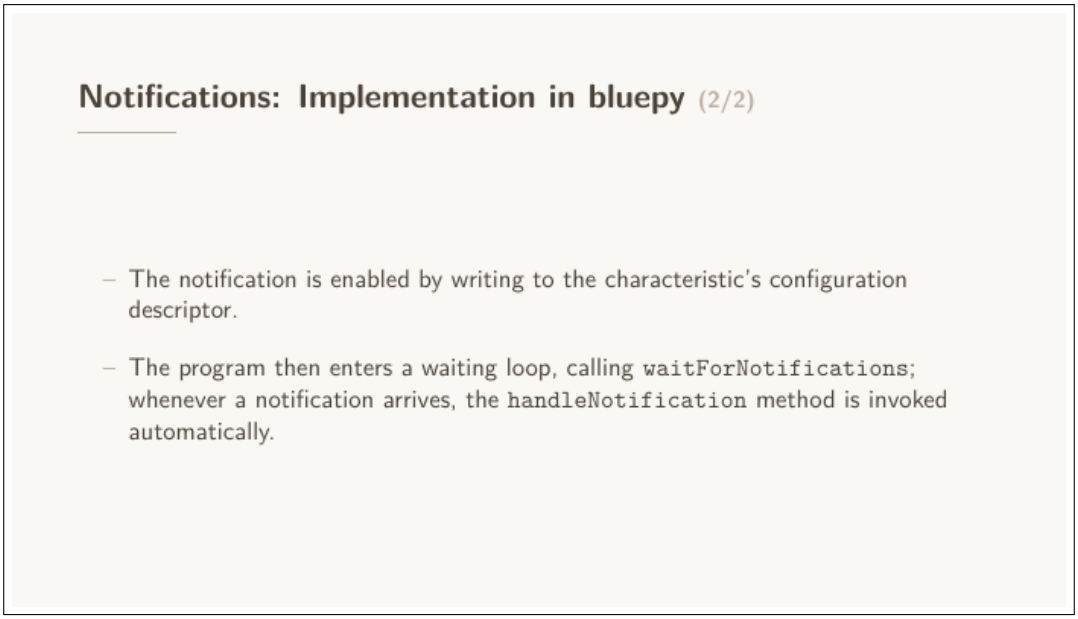
The three operations and their cost. The diagram compares read, write and notification. In the first two the client takes the initiative. In the third, the server sends without being asked. The footnote summarises the essential: periodic polling consumes energy even when nothing changed, whereas notification only transmits when there is news.

Slide 17. Notifications: Implementation in bluepy (1/2)



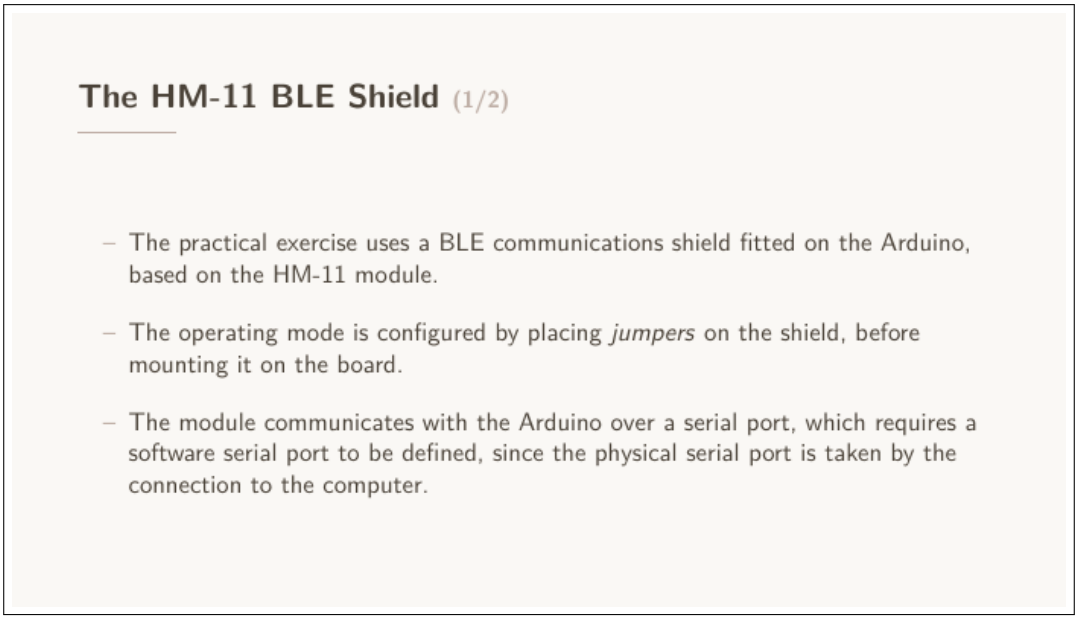
The implementation steps. A subclass of DefaultDelegate is defined, overriding handleNotification, which receives the characteristic's handle and the data. The handle lets the origin be distinguished when several characteristics notify. The Peripheral object is created, the delegate attached, the service obtained with getServiceByUUID and the characteristic with getCharacteristics.

Slide 18. Notifications: Implementation in bluepy (2/2)



Activation and the wait loop. Notification is enabled by writing to the characteristic's configuration descriptor. This step is often forgotten and is the most common cause of receiving nothing. The program then enters a loop that calls waitForNotifications. Whenever a notification arrives, handleNotification is invoked automatically.

Slide 19. The HM-11 BLE Shield (1/2)



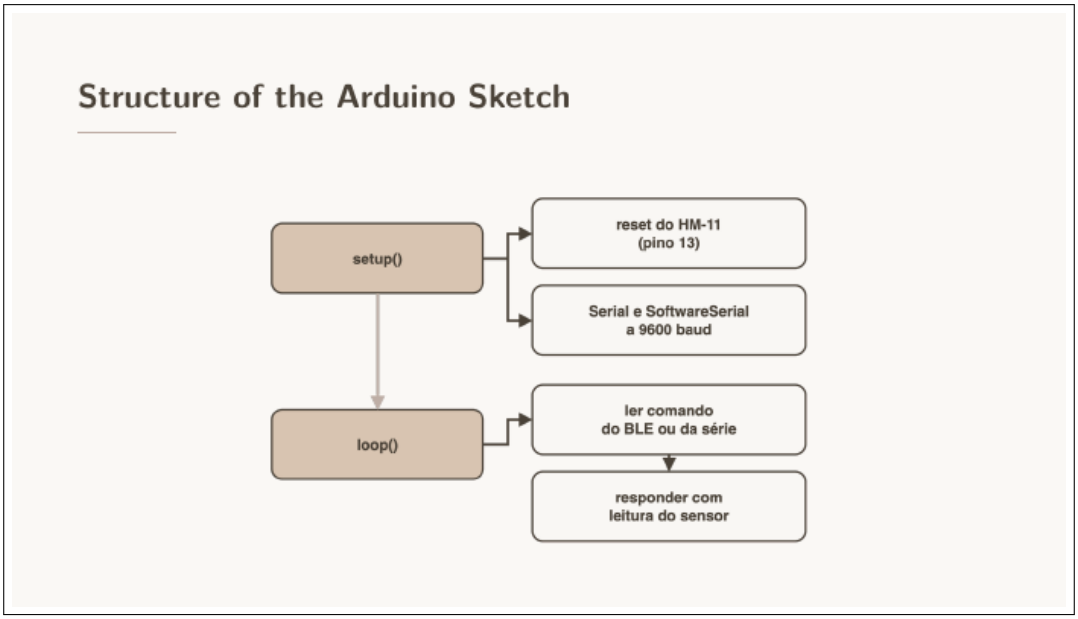
The HM-11 module. The exercise uses a shield mounted on the Arduino, based on the HM-11 module. Mode configuration is done by jumpers, before assembly. The module communicates with the Arduino through a serial port, which requires a software serial port to be defined, since the hardware one is occupied by the connection to the computer.

Slide 20. The HM-11 BLE Shield (2/2)



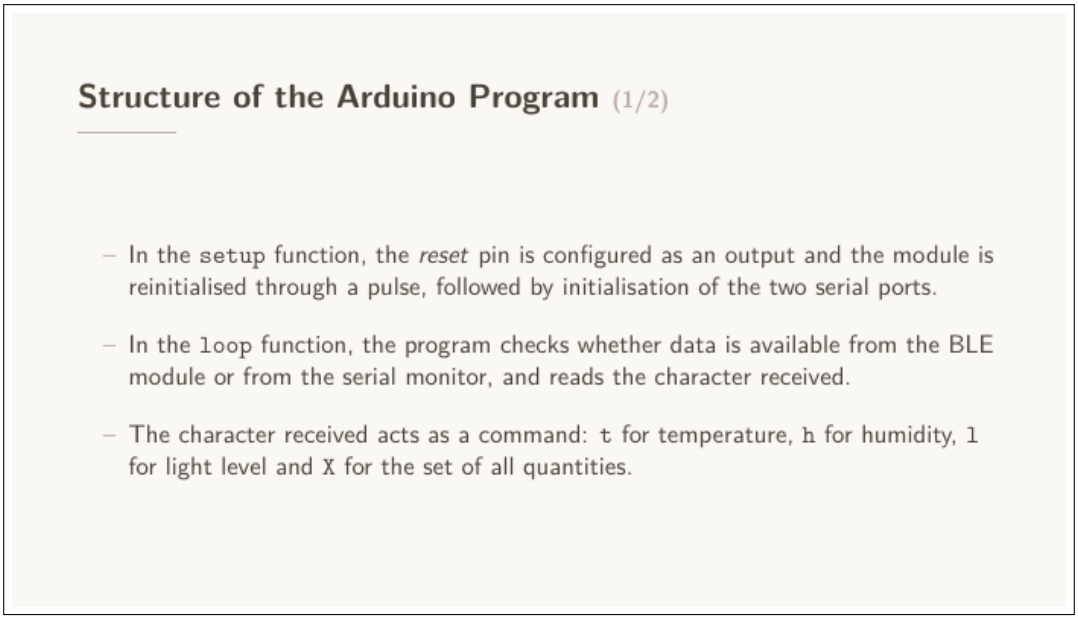
Reset and baud rates. The module's reset pin is controlled by a digital output, which lets the module be reinitialised at program start. Both serial ports operate at 9600 bit/s. Mismatches in this value produce unreadable text, and are a frequent source of error in the lab.

Slide 21. Structure of the Arduino Sketch



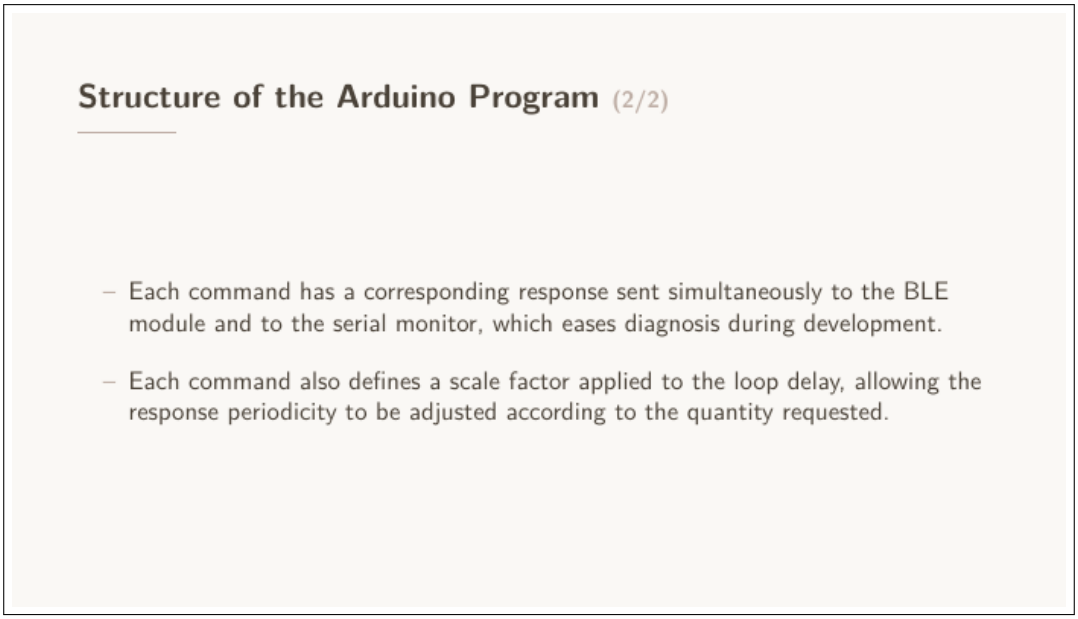
The program structure, in a diagram. The diagram separates what happens once, in setup, from what repeats in loop. In setup, module reset and opening of the two ports. In loop, reading a command and answering with the sensor reading.

Slide 22. Structure of the Arduino Program (1/2)



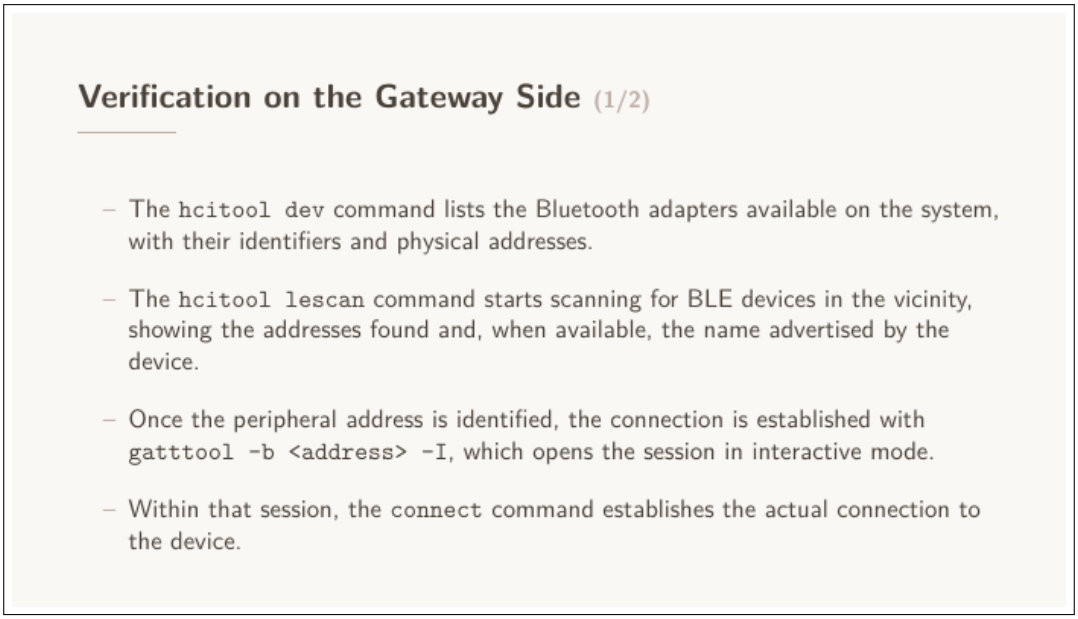
What setup does and what loop does. In setup the reset pin is configured, the module is reset with a pulse and the two serial ports are opened. In loop it checks whether there is data from the BLE module or from the serial monitor. The character received works as a command: t for temperature, h for humidity, l for luminosity and X for the set.

Slide 23. Structure of the Arduino Program (2/2)



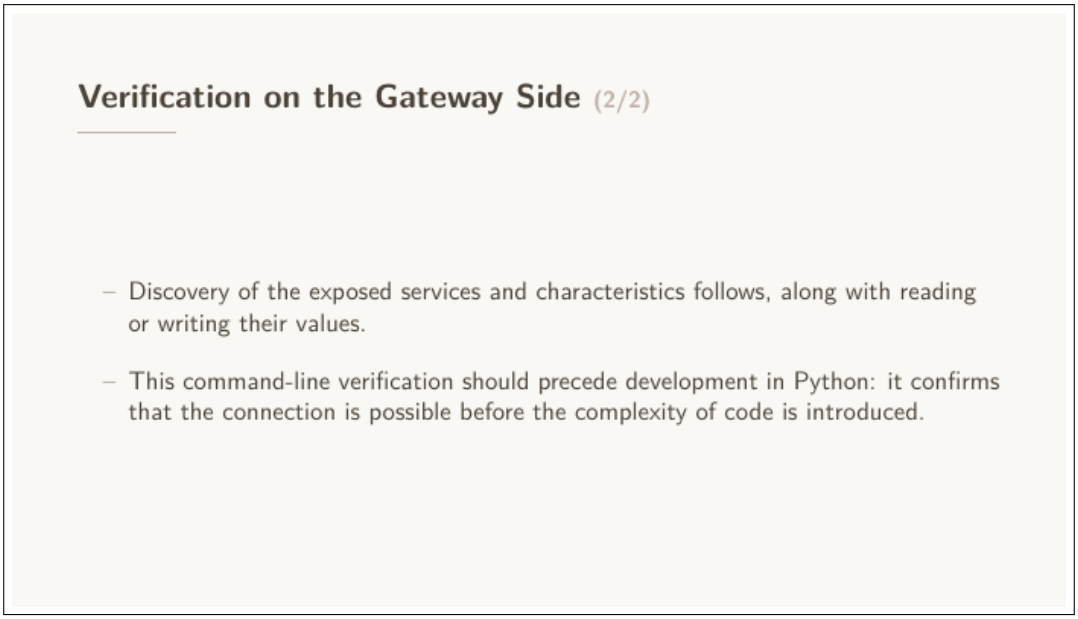
A design decision that eases diagnosis. The reply is sent simultaneously to the BLE module and to the serial monitor. This duplication is deliberate and lets the sketch's operation be checked without involving BLE communication. Each command also defines a scale factor applied to the cycle delay.

Slide 24. Verification on the Gateway Side (1/2)



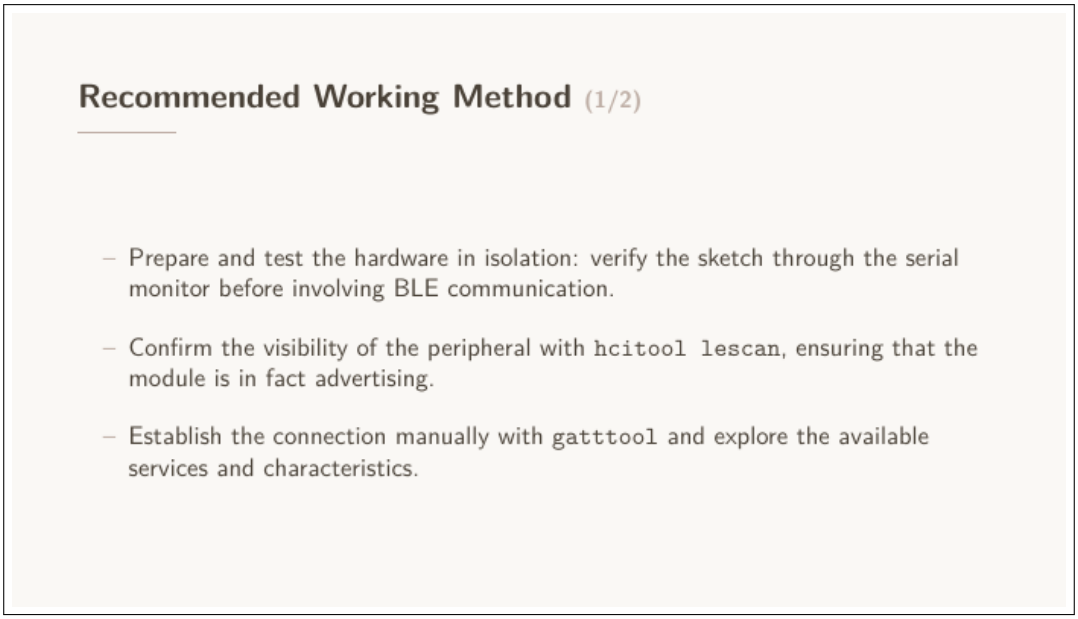
Verify before programming. The command hcitool dev lists adapters. hcitool lescan looks for devices. Once the address is identified, the connection is established with gatttool in interactive mode, and within that session the connect command actually connects.

Slide 25. Verification on the Gateway Side (2/2)



The reason for the order. The discovery of services and characteristics follows, and then reading or writing the values. The justification given on the slide is the point to retain: this verification should precede development in Python, because it confirms that the connection is possible before introducing the complexity of code.

Slide 26. Recommended Working Method (1/2)



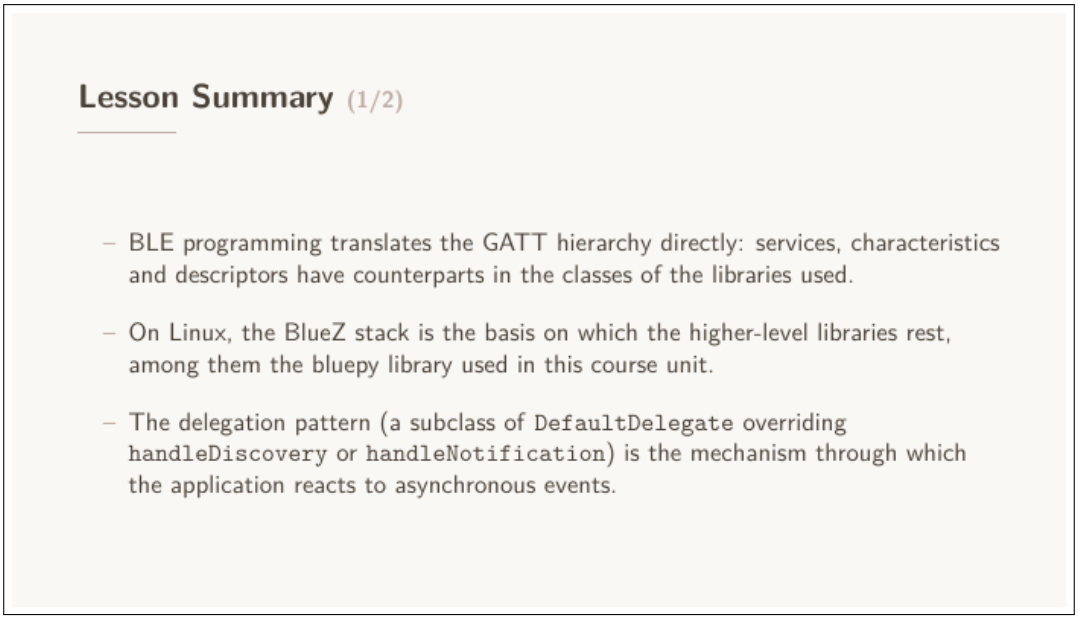
The first three steps. Test the hardware in isolation through the serial monitor. Confirm the peripheral's visibility with hcitool lescan. Establish the connection manually with gatttool and explore the services.

Slide 27. Recommended Working Method (2/2)



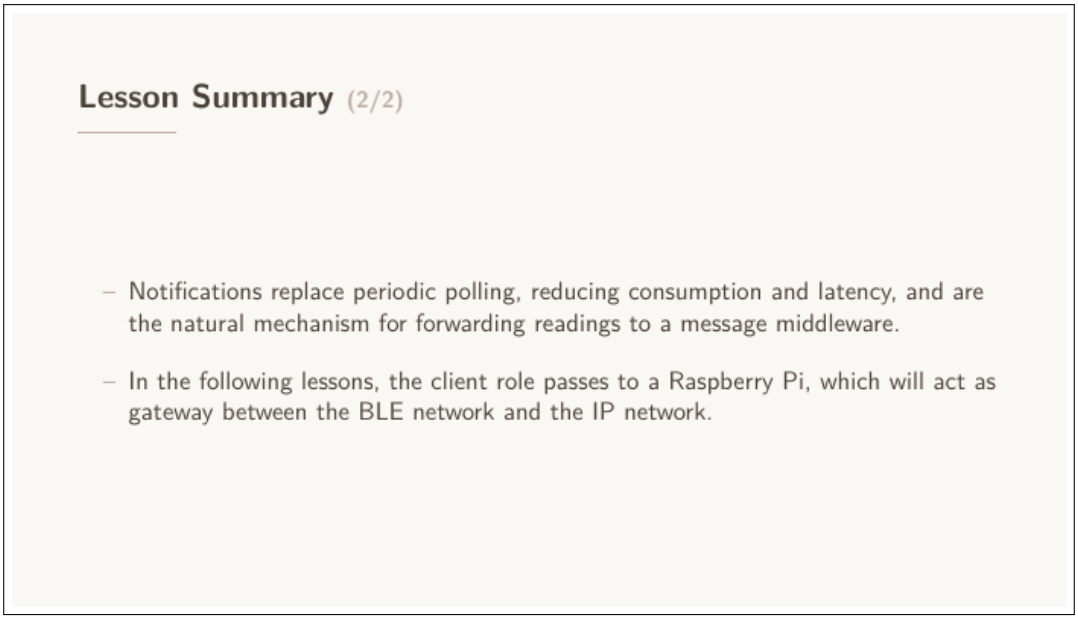
Why the progression matters. Only then move on to development in Python, from scanning to reading and to notifications. The justification is methodological and generalisable: this progression isolates the origin of problems. A failure detected at the third step cannot be attributed to the Python code, which does not yet exist.

Slide 28. Lesson Summary (1/2)



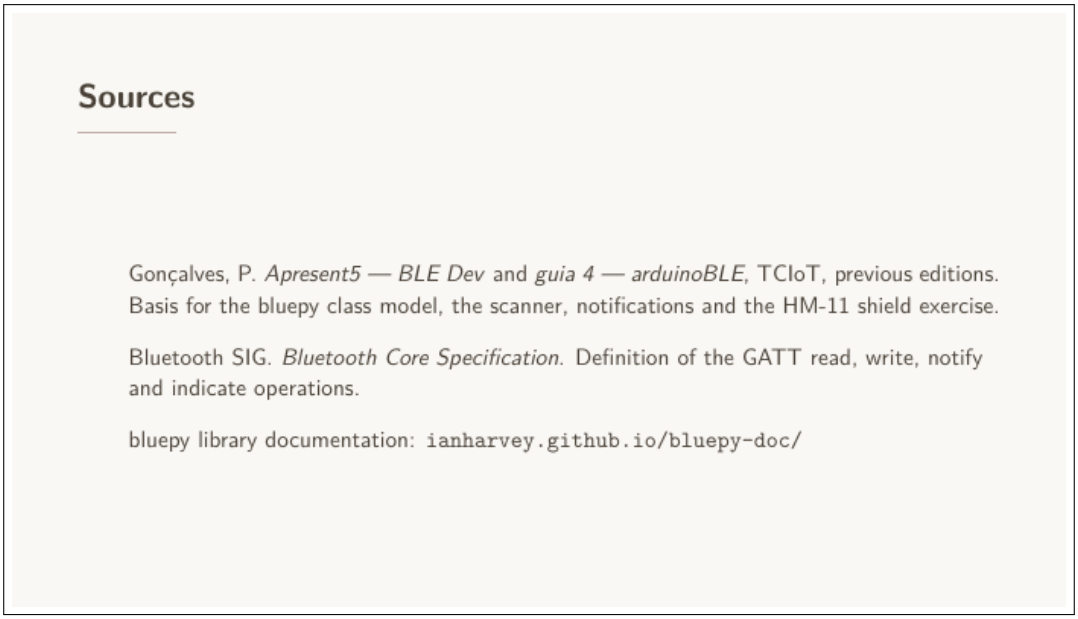
Summary, first part. BLE programming translates the GATT hierarchy directly. On Linux, BlueZ is the base on which the higher-level libraries sit. The delegation pattern, with a DefaultDelegate subclass and override of handleDiscovery or handleNotification, is the mechanism by which the application reacts to asynchronous events.

Slide 29. Lesson Summary (2/2)



Summary, second part. Notifications replace periodic polling, reducing consumption and latency, and are the natural mechanism for forwarding readings to a message middleware. In the following lectures the client role passes to a Raspberry Pi, which will act as a gateway between the BLE network and the IP network.

Slide 30. Sources



Origin of the material. The basis is the course material from previous editions, complemented by the Bluetooth SIG specification on GATT operations and by the bluepy documentation.

Final Check

Before considering the material studied, check that you can answer the following without looking at the slides.

1. Explain the correspondence between the bluepy classes and the GATT hierarchy of Lecture 04.
2. Describe the four steps needed to implement a scanner.
3. Distinguish notification from indication and state which of them requires acknowledgement from the client.
4. State the activation step without which no notification is received.
5. Justify the need for a software serial port on the Arduino.
6. Explain why verification with gatttool should precede development in Python.
7. State what can and cannot be concluded from an RSSI value.

References

- Gonçalves, P. *Apresent5 — BLE Dev* and *guide 4 — arduinoBLE*, TCIoT, previous editions.
- Bluetooth SIG. *Bluetooth Core Specification*.
- bluepy library documentation: `ianharvey.github.io/bluepy-doc/`